

Knowledge Inference and Data Mining with Formal Concept Analysis: Interpretable Heart Disease Diagnosis on the Cleveland Dataset



Additional Materials

MSc in Data Science (Computer Science Data Informatics)

DATA72000

Table of Contents:

1. Code Repository:	2
2. Files inside Repository:	2
3. Software Environment:	3
4. Installing required Softwares:	3
4.1 macOS	3
4.2 Linux	4
4.3 Windows	4
5. Obtaining FCA4J library:	4
6. Accessing the Dataset:	4
7. Running the project:	5

8. Output Files produced:.....6

This document supports the Extended Research Project Report in reproducing the project results. It includes exact instructions, explains the files required and produced, lists the software and command prompts for ease of execution.

1. Code Repository:

All source code is uploaded in this public github repository:

Link:

<https://github.com/NusaibaZaman/Knowledge-Inference-and-Data-Mining-with-Formal-Concept-Analysis.git>

2. Files inside Repository:

These files are required for reproducibility.

File	Role
clean_data.py	Reads processed.cleveland.data, removes incomplete records, writes heart_clean.csv and targets.txt
binarise.py	Binarisation module holding the clinical and entropy cut points, imported by build_contexts.py and compare_models.py
build_contexts.py	Reads heart_clean.csv and writes the clinical and entropy formal contexts
feature_risk.py	Reads a context and targets.txt, computes each attribute's odds ratio and Fisher's exact test and draws the risk figure
compare_models.py	Runs the five-fold comparison of the FCA classifier against the baselines and writes the summary and per-patient score files
summary_table.py	Reads the results and summary files and writes the combined summary table
pom.xml	Maven build file, declares the fca4j dependency and the main class

File	Role
src/main/java/Lattice.java	Builds a lattice from a context and mines and prunes the diagnostic rules
src/main/java/Classify.java	Scores patients from a training context, called by compare_models.py inside each fold
src/main/java/ExampleLattice.java	Builds the small worked-example lattice used for the methodology figure
context_example.csv	The small worked-example formal context (input for ExampleLattice)
targets_example.txt	The labels for the worked example

Table 1: Lists the repository files and explains what each file does

3. Software Environment:

The project was built and run on macOS with the versions below.

Component	Version	Role in the project
Python	3.11.14	Runs the cleaning, binarisation, risk, comparison and summary scripts
pandas	2.2.2	Reads and reshapes the tabular data
NumPy	1.26.4	Numerical arrays used across the Python scripts
scikit-learn	1.8.0	Baseline classifiers and the cross-validation split
SciPy	1.17.1	Fisher's exact test for the attribute statistics
Matplotlib	3.10.6	Draws the attribute risk figures
Java (OpenJDK)	21.0.6	Runs the lattice construction and rule mining
Apache Maven	3.9.16	Builds the Java code into a runnable jar
fca4j	0.4.7	Concept lattice construction and rule mining (LIRMM)
Graphviz (dot)	12.2.1	Renders the worked-example lattice to an image

Table 2: Software Versions used

The fca4j library is not on Maven Central and this project's pom.xml does not declare a repository for it, thus it is not downloaded automatically when the Java code is built. It must be placed in the local Maven repository first, which Section 5 explains.

4. Installing required Softwares:

The project requires five tools: a Java Development Kit (version 17 or newer), Apache Maven, Python 3, Graphviz and Git.

4.1 macOS

Step 1: Install Homebrew

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 2: Install Java, Maven, Python, Graphviz and Git

```
brew install openjdk@21 maven python@3.11 graphviz git
```

Step 3: Install the Python libraries

```
pip3 install pandas numpy scikit-learn scipy matplotlib
```

4.2 Linux

```
sudo apt update
sudo apt install default-jdk maven python3 python3-pip graphviz git
pip3 install pandas numpy scikit-learn scipy matplotlib
```

4.3 Windows

```
winget install Microsoft.OpenJDK.21
winget install Apache.Maven
winget install Python.Python.3.11
winget install Graphviz.Graphviz
winget install Git.Git

python -m pip install pandas numpy scikit-learn scipy matplotlib
```

5. Obtaining FCA4J library:

The Java build depends on `fr.lirmm.fca4j:fca4j-io` version 0.4.7. This is a research library from LIRMM, not a package on Maven Central, it has to be built once and installed into the local Maven repository before the project will build.

Clone the LIRMM fca4j project and install it. This places fca4j-io 0.4.7, along with the other fca4j modules it needs, into the local Maven repository.

```
git clone https://gite.lirmm.fr/gutierre/fca4j-project.git
cd fca4j-project
mvn clean install
cd ..
```

Version note: the project's pom.xml asks for 0.4.7. If the LIRMM project's current version is different, check the 0.4.7 release before installing, or change the version number in this project's pom.xml to match the version currently.

6. Accessing the Dataset:

The project uses the Cleveland subset of the UCI Heart Disease dataset. It has been downloaded from:

Link: <https://archive.ics.uci.edu/dataset/45/heart+disease>

After downloading the dataset file, select the processed.cleveland.data file and place it in the project root. The clean_data.py file reads the dataset directly, it drops the records with missing values and writes the cleaned dataset that the later steps use.

7. Running the project:

After cloning the github repository, run these steps exactly to reproduce the results:

Step 1: Install all python packages

```
pip install pandas numpy scikit-learn scipy matplotlib
```

Step 2: Build the Java tools

```
mvn clean package
```

Step 3: Clean the raw dataset

```
python3 clean_data.py
```

Step 4: Build the Formal Contexts (both Clinical and Entropy contexts)

```
python3 build_contexts.py
```

Step 5: Build the Lattice and mine the rules

```
java -cp target/<jar-name>.jar Lattice context_clinical.csv 30 0.70 targets.txt >
results_clinical.txt
```

```
java -cp target/<jar-name>.jar Lattice context_entropy.csv 30 0.70 targets.txt >
results_entropy.txt
```

Step 6: Compute the attribute risk statistics

```
python3 feature_risk.py context_clinical.csv
```

```
python3 feature_risk.py context_entropy.csv
```

Step 7: Compare FCA Classifier with Machine Learning models

```
python3 compare_models.py clinical
```

```
python3 compare_models.py entropy
```

Step 8: Build Combines summary table

```
python3 summary_table.py
```

Step 9: Render the worked example lattice

```
java -cp target/<jar-name>.jar ExampleLattice context_example.csv example_lattice.dot
```

```
dot -Tpng example_lattice.dot -o figure_3_2_lattice_example.png
```

8. Output Files produced:

These files will be produced as outputs after Section 7.

Output file	Produced by	Contains
heart_clean.csv	clean_data.py	The cleaned 297-patient dataset (13 features and the target)
targets.txt	clean_data.py	Each patient id with its 0 or 1 label, used by Lattice and feature_risk.py
context_clinical.csv	build_contexts.py	The clinical binarised formal context

Output file	Produced by	Contains
context_entropy.csv	build_contexts.py	The entropy binarised formal context
results_clinical.txt	Lattice (clinical)	Lattice size and the mined and pruned diagnostic rules (clinical)
results_entropy.txt	Lattice (entropy)	Lattice size and the mined and pruned diagnostic rules (entropy)
feature_risk_output_clinical.csv	feature_risk.py	Per-attribute odds ratio, p-value and patient counts (clinical)
feature_risk_output_entropy.csv	feature_risk.py	Per-attribute odds ratio, p-value and patient counts (entropy)
figure_feature_risk_clinical.png	feature_risk.py	Odds-ratio bar chart, clinical (Figure 4a in the report)
figure_feature_risk_entropy.png	feature_risk.py	Odds-ratio bar chart, entropy (Figure 4b in the report)
summary_clinical.csv	compare_models.py	FCA versus best baseline accuracy and AUC (clinical)
summary_entropy.csv	compare_models.py	FCA versus best baseline accuracy and AUC (entropy)
oof_scores_clinical.csv	compare_models.py	The out-of-fold disease score for each patient (clinical)
oof_scores_entropy.csv	compare_models.py	The out-of-fold disease score for each patient (entropy)
summary_table.csv	summary_table.py	Combined lattice size, rule count and predictive metrics
example_lattice.dot	ExampleLattice	Graphviz description of the worked-example lattice
figure_3_2_lattice_example.png	dot (Graphviz)	The rendered worked-example lattice (Figure 2 in the report)
fca-skeleton-1.0-jar-with-dependencies.jar	mvn clean package	The runnable Java tool, with fca4j bundled inside

Table 3: The Output files produced after running the files in the repository